

Documentazione progetto dimostrazione assistita

Dimitri B.

21 settembre 2026

1 Introduzione

L'obiettivo del progetto è l'implementazione di un sistema di tipaggio che presenta le regole standard e viene ampliato con l'espressione *if*. Attraverso le regole definite bisogna poi dimostrare tre teoremi e una proposizione forniti.

Nel capitolo che segue (2) presenteremo la struttura generale dell'elaborato e le principali scelte di implementazione. Seguirà poi (3) una spiegazione delle dimostrazioni e dei lemmi utilizzati per portare a termine queste ultime.

2 Struttura generale

Nella prima sezione dell'elaborato riportiamo la definizione induttiva del tipo T e del termine t . Il termine t è definito in modo tale da poter assumere diverse forme, tra cui: variabili, costanti booleane, funzioni, espressione IF e l'applicazione. In questa sezione definiamo solo il tipo dei vari costrutti, ossia quali e quanti valori sono richiesti per computare il risultato (di cui viene fornito il tipo).

Segue poi la definizione di *val*, ossia espressioni che, come i valori booleani false/true e le astrazioni (funzioni), hanno terminato la loro computazione e non possono essere ulteriormente ridotte.

Procediamo con la definizione della funzione ricorsiva **subst**, implementazione della sostituzione di un termine s in ogni variabile x nel termine t , a cui d'ora in poi faremo riferimento con la notazione $t[x/s]$. Ci serviamo della funzione **eqb** proveniente dalla libreria *String* per verificare se x e una qualsiasi variabile y risultino coincidere in modo da poter applicare correttamente le regole della sostituzione.

Tramite le definizioni induttive **single_step** e **multi_step**, otteniamo la riduzione $\longrightarrow$ (ossia l'evoluzione di un termine in seguito a un solo passo di valutazione) e la sua chiusura transitiva e riflessiva $\longrightarrow^*$, a cui faremo riferimento con la notazione $\Rightarrow$ e $\Rightarrow^*$.

Nella relazione **single_step** abbiamo specificato diverse regole di inferenza; una tra queste

è la beta-riduzione, dove, avendo come valore l'argomento di un'applicazione di una funzione, possiamo ridurla a una sostituzione del suddetto valore al posto di una variabile x nell'espressione e . La relazione **multi_step** invece descrive una sequenza di passi multipli. In particolare, la regola **ms_step** descrive una concatenazione di un passo singolo con una riduzione multipla successiva.

Concludiamo le definizioni con il *type system*. Il contesto di tipizzazione Γ è stato implementato come una lista di coppie (stringa, T).

La funzione ausiliaria **find_type** esegue la ricerca ricorsiva del tipo associato a una variabile x all'interno di Γ : scorre la lista elemento per elemento e restituisce **None** qualora la variabile non sia presente nel contesto (incluso il caso di contesto vuoto).

Il *type system* vero e proprio, a cui ci riferiamo mediante la notazione $\Gamma \vdash e :: T$, è stato definito induttivamente tramite le classiche regole di inferenza (rappresentate in Rocq dai costruttori della relazione **type_sys**).

Durante le numerose definizioni fatte, sono state specificate anche diverse notazioni, come:

- **A $\rightarrow$ B** per fare riferimento al tipo *arrow_type A B*
- **Fun(a : type){e}** per riferirci alla funzione
- **App(term1, term2)** per l'applicazione
- **If(a){b otherwise c}** per il costrutto IF
- **Find x within {gamma}** per la ricerca in un contesto
- **v is a value** per i valori
- **[]** per il contesto vuoto

Consideriamo infine la definizione di *programma*: considerando i programmi come termini privi di variabili libere, la semantica CBV elimina i fenomeni di cattura delle variabili poiché essendo il termine v chiuso, una volta ridot-

to a valore, manterrà questa sua caratteristica; evitando così catture di variabili in sostituzioni.

3 Dimostrazioni

3.1 Teorema 1

$$\neg(\exists S, T. \vdash \lambda x : S. x x : T)$$

Questo primo teorema esprime la non tipabilità dell'applicazione di una funzione a se stessa. I primi passi della dimostrazione servono a ricondurre il teorema a una forma più semplice che metta in risalto la relazione di uguaglianza (impossibile) tra i tipi. Una volta ottenuta l'ipotesi:

$$H_eq : ty_2 = ty_2 - - > ty_1$$

Per dimostrare l'impossibilità di tale uguaglianza, si applica la funzione ausiliaria **size** ad entrambi i lati tramite il lemma **f_equal**. Otteniamo così un'uguaglianza sulla profondità del tipo che risulta banale dimostrare essere impossibile.

3.2 Teorema 2

$$\exists T, [] \vdash \lambda x : \text{Bool} \rightarrow \text{Bool}. \lambda y : \text{Bool} \rightarrow \text{Bool}.$$

$$\lambda z : \text{Bool}. y(xz) : T$$

L'obiettivo della dimostrazione è verificare la correttezza del tipo per il termine lambda espressivo della composizione di funzioni applicata a un argomento (assumo che il precedente tipo $\text{Bool} \rightarrow B$ fosse un errore di battitura e che mancasse l'esistenziale poiché altrimenti non sarebbe tipabile). Tramite la tattica **exists**, forniamo esplicitamente il tipo dell'applicazione. Seguono poi diverse applicazioni della regola **lambda_r** con lo scopo di aggiungere x , y e z al contesto Γ . Dividiamo quindi la dimostrazione in due sotto-goals tramite l'applicazione della regola **app_r**.

Il primo è dimostrabile poiché nel contesto Γ è presente la variabile y tipata come **Bool** $\rightarrow$ **Bool**, sostituendola quindi nell'equazione obiettivo, ci basterà applicare la riflessività per concludere. Per raggiungere questo scopo ci serviamo della sequenza di tattiche:

apply base_r; simpl; reflexivity.

La dimostrazione del secondo goal procede similmente, con la differenza che ci è richiesto suddividere ulteriormente la dimostrazione in altri due goals, per poi approcciarli (caso x e caso z) singolarmente come precedentemente illustrato.

Rocq prover mette a disposizione dell'utente lo strumento **constructor**, una tattica capace di

applicare il costruttore necessario per la risoluzione del goal automaticamente. Il teorema è dimostrabile servendosi di questo strumento con la sola riga

repeat econstructor

3.3 Proposizione 1

$$(\lambda x : \text{Bool} \rightarrow \text{Bool}. x)$$

$$((\lambda x : \text{Bool}. \text{if } x \text{ then false else true}) \text{ true})$$

$$\longrightarrow^* \text{false}$$

Questa proposizione fornisce un esempio pratico della semantica operativa definita in precedenza, illustrando la valutazione multi-step di un'applicazione annidata.

Incominciamo la dimostrazione introducendo un termine intermedio tra il blocco principale e il valore booleano "false", tramite l'applicazione di **ms_step**. Consideriamo quindi il primo dei due goals: una riduzione a singolo passo dal corpo principale a un termine intermedio t_2 .

Dato che il termine presenta un'applicazione, ci troviamo costretti a dimostrare la natura di valore del primo termine e la riduzione del secondo, situazione gestita facilmente dalla tattica **econstructor**.

Passiamo ora al secondo goal, il quale presenta la riduzione multipla. Svolgendo la sostituzione (con **simpl**) ci troviamo costretti a procedere con la riduzione del termine principale applicando nuovamente **ms_step**. Ci ritroviamo quindi in una situazione simile alla precedente, continuiamo ad avanzare nella riduzione seguendo un procedimento speculare al sopracitato.

Questa serie di passaggi termina collassando l'intera riduzione multi-step in un semplice caso riflessivo.

3.4 Teorema di Preservazione del Tipo (Subject Reduction)

$$\forall t, t', T. (\vdash t : T \wedge t \longrightarrow^* t') \implies \vdash t' : T$$

Concludiamo infine con il teorema di **Subject Reduction**, il quale, come espresso formalmente qua sopra, stabilisce la preservazione del tipo in seguito a riduzione. Per dimostrare tale risultato ci serviamo del lemma di sostituzione:

$$(\Gamma, x : U \vdash t : T) \wedge ([] \vdash v : U) \implies (\Gamma \vdash t[x/v] : T)$$

Una dimostrazione comune vede questo teorema dimostrato tramite il lemma di Weakening, secondo il quale:

$$(\Gamma \vdash t : T) \implies (\Gamma, x : A \vdash t : T).$$

(ammesso che x non sia fresh).

In questo progetto ci serviremo della seguente formulazione del lemma, pensata per semplificare i passaggi:

$$\begin{aligned} (\Gamma_1 \vdash t : T) &\implies \\ (\forall y, A. \text{find_type}(y, \Gamma_1) = \text{Some } A \implies \\ \text{find_type}(y, \Gamma_2) = \text{Some } A) &\implies \\ (\Gamma_2 \vdash t : T). \end{aligned}$$

Le dimostrazioni di questi lemmi sfruttano un approccio più automatizzato rispetto alle precedenti, grazie alle tattiche **eauto** ed **econstructor**, che forniscono un'alternativa più sintetica alle dimostrazioni tradizionali.

3.4.1 Pre-dimostrazioni

- **Primo lemma (sub_context):** Stabilisce che, dati due contesti Γ_1 e Γ_2 , se $\Gamma_1 \vdash e :: T$ e una ricerca del tipo di una qualsiasi variabile in Γ_1 produce un tipo U , la stessa operazione su Γ_2 deve produrre il medesimo risultato, allora $\Gamma_2 \vdash e :: T$. In altre parole stiamo chiedendo che $\Gamma_1 \subseteq \Gamma_2$. La dimostrazione procede per induzione sull'ipotesi di tipizzazione $H : \Gamma_1 \vdash e :: T$. L'uso della combinazione:

econstructor; eauto.

risolve automaticamente i casi strutturali più semplici, lasciando scoperto un solo *subgoal*, risolvibile applicando l'ipotesi induttiva e semplificando.

- **Secondo lemma (empty_context):** Deriva direttamente come conseguenza del precedente.
- **Terzo lemma (substitution_preserves_type):** Sancisce che la sostituzione di una variabile con un termine ben tipato preserva il tipo dell'espressione originaria. La dimostrazione procede per induzione sulla struttura dell'espressione e . I casi base ed elementari vengono risolti automaticamente da Rocq; restano da trattare in modo specifico due scenari:

Nel caso della variabile, scomponendo l'uguaglianza dei nomi tramite l'uso dell'asserzione **String.eqb_spec**, ci si riduce a due sottocasi. Nel primo si osserva l'uguaglianza dei tipi U e T , applicando il lemma di *empty_context* previa sostituzione; il secondo sottocaso è banale.

Infine nel caso della funzione, si scompone analogamente la condizione sui nomi dei legami. Entrambi i sottocasi si avvalgono del lemma **sub_context** per essere risolti, ma per motivi differenti, nel primo dato che $s = x$, dimostriamo che:

$$((x, t_0) :: \Gamma) \subseteq ((x, t_0) :: (x, U) :: \Gamma)$$

mentre nel secondo che:

$$((x, U) :: (s, t_0) :: \Gamma) = ((s, t_0) :: (x, U) :: \Gamma)$$

3.4.2 Teorema 4

Fatte dunque le dovute pre-dimostrazioni, passiamo ora al teorema finale di **subject reduction**. La dimostrazione procede per induzione sulla relazione di riduzione multi-step $\implies^> *$. Risolto il caso base, affrontiamo quello induttivo, dove applicando l'ipotesi induttiva, ci troviamo con il goal $t_2 :: T_0$ riducendo quindi il problema a dimostrare che la preservazione del tipo valga per un singolo passo di riduzione. Una volta generalizzato per T_0 otteniamo in:

$$\forall T_0 : T, [] \vdash t_1 :: T_0 \implies [] \vdash t_2 :: T_0$$

dimostrato per induzione sulla riduzione $t_1 \implies^> t_2$.

Considerando il primo e più importante caso, la sostituzione, tramite l'inversione dell'ipotesi induttiva ricaviamo che il termine e risulta essere di tipo T_0 e che il termine volto a sostituire la variabile x è dello stesso tipo ($type_2$). Queste sono le condizioni necessarie per applicare il teorema della sostituzione precedentemente dimostrato e concludere questo sotto-goal.

Il resto della dimostrazione risulta seguire lo schema presentato in questo primo caso ad eccezione dell'applicazione del teorema.

Conclusione

L'elaborato, grazie alla costruzione di regole e fondamentali di un linguaggio standard semplicemente tipato, mi ha permesso di visualizzare una rappresentazione grafica e algoritmica delle dimostrazioni, permettendomi di comprenderle più a fondo.